



Shaheed Zulfikar Ali Bhutto
Institute of Science & Technology

Department of Computer Science

Course: Database Management System Lab

Instructor: Sir Abid Ali

BSCS 4A

Project

“BITE DASH-ONLINE FOOD DELIVERY SYSTEM”

Group Members:

- M Hamza Bhatti (2312122)
- M Moeed (2312396)

Table of Contents

S.No	Title	Page Number
1	Introduction	3
2	Objectives	4
3	Database tables	5 - 12
4	ERD Diagram	13
5	Core Functionalities	14 - 17
6	APIs	18 - 22
7	Front-End screenshots	23 - 24
8	Conclusion	25

Introduction

Online food delivery systems have become a fundamental part of our lifestyles. The Online Food Delivery System developed in this project provides a robust platform for customers to browse menus, place orders, and receive their food from multiple restaurants through a seamless interface. This system is designed to streamline the food ordering process for customers while also offering restaurants a platform to manage their menus, track orders, and analyze customer reviews.

The backend infrastructure is powered by PostgreSQL hosted on Neon.tech, ensuring reliable and scalable data handling. The backend APIs are built using Node.js and Express.js, offering RESTful endpoints for full CRUD operations. The frontend is implemented with HTML, CSS, and JavaScript, delivering a responsive and user-friendly experience. From database normalization to data visualization, this project covers a wide range of database and web technologies.

Objectives

The main goals of the Online Food Delivery System project are outlined as follows:

- • To create a well-structured and normalized relational database that handles all aspects of an online food delivery system, including users, restaurants, orders, and delivery.
- • To implement secure and scalable APIs using Node.js and Express.js for performing database operations like retrieving, inserting, and deleting records.
- • To build a visually appealing and functional frontend interface that interacts with the backend to display live data and manage user interactions.
- • To demonstrate real-world functionalities such as menu browsing, order placement, payment recording, delivery tracking, and review posting within a full-stack application.
- • To allow customers to manage multiple addresses and enable restaurants to belong to multiple food categories for flexible search and filtering.

Database tables

There are 12 tables in online food ordering system database.

Table Prompt:

```
CREATE TABLE Customers (  
    customer_id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    phone VARCHAR(20)  
);  
  
CREATE TABLE Addresses (  
    address_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES Customers(customer_id),  
    street VARCHAR(255),  
    city VARCHAR(100),  
    zip_code VARCHAR(20),  
    label VARCHAR(50)  
);  
  
CREATE TABLE Restaurants (  
    restaurant_id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    phone VARCHAR(20),  
    address TEXT  
);
```

```
CREATE TABLE Food_Categories (  
    category_id SERIAL PRIMARY KEY,  
    category_name VARCHAR(100)  
);
```

```
CREATE TABLE Menu_Items (  
    item_id SERIAL PRIMARY KEY,  
    restaurant_id INT REFERENCES Restaurants(restaurant_id),  
    category_id INT REFERENCES Food_Categories(category_id),  
    item_name VARCHAR(100),  
    description TEXT,  
    price DECIMAL(10,2),  
    availability BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE Orders (  
    order_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES Customers(customer_id),  
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(50),  
    total_amount DECIMAL(10,2)  
);
```

```
CREATE TABLE Order_Items (  
    order_item_id SERIAL PRIMARY KEY,
```

```

    order_id INT REFERENCES Orders(order_id),
    item_id INT REFERENCES Menu_Items(item_id),
    quantity INT,
    price DECIMAL(10,2)
);

CREATE TABLE Payments (
    payment_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES Orders(order_id),
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    amount DECIMAL(10,2),
    payment_method VARCHAR(50),
    payment_status VARCHAR(50)
);

CREATE TABLE Delivery_Agents (
    agent_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    phone VARCHAR(20),
    vehicle_number VARCHAR(50)
);

CREATE TABLE Deliveries (
    delivery_id SERIAL PRIMARY KEY,
    order_id INT REFERENCES Orders(order_id),
    agent_id INT REFERENCES Delivery_Agents(agent_id),

```

```
delivery_status VARCHAR(50),  
delivery_time TIMESTAMP  
);
```

```
CREATE TABLE Reviews (  
    review_id SERIAL PRIMARY KEY,  
    customer_id INT REFERENCES Customers(customer_id),  
    restaurant_id INT REFERENCES Restaurants(restaurant_id),  
    rating INT CHECK (rating BETWEEN 1 AND 5),  
    comment TEXT,  
    review_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE Restaurant_Categories (  
    restaurant_id INT REFERENCES Restaurants(restaurant_id),  
    category_id INT REFERENCES Food_Categories(category_id),  
    PRIMARY KEY (restaurant_id, category_id)  
);
```



```
project=> \dt
```

List of relations			
Schema	Name	Type	Owner
public	addresses	table	project_owner
public	customers	table	project_owner
public	deliveries	table	project_owner
public	delivery_agents	table	project_owner
public	food_categories	table	project_owner
public	menu_items	table	project_owner
public	order_items	table	project_owner
public	orders	table	project_owner
public	payments	table	project_owner
public	restaurant_categories	table	project_owner
public	restaurants	table	project_owner
public	reviews	table	project_owner

(12 rows)

```

project=> \d addresses

      Table "public.addresses"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
address_id | integer                |           | not null | nextval('addresses_address_id_seq'::regclass)
customer_id | integer                |           |           |
street      | character varying(255) |           |           |
city        | character varying(100) |           |           |
zip_code    | character varying(20)  |           |           |
label       | character varying(50)  |           |           |
Indexes:
    "addresses_pkey" PRIMARY KEY, btree (address_id)
Foreign-key constraints:
    "addresses_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)

project=> \d customers

      Table "public.customers"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
customer_id | integer                |           | not null | nextval('customers_customer_id_seq'::regclass)
name        | character varying(100) |           |           |
email       | character varying(100) |           |           |
phone       | character varying(20)  |           |           |
Indexes:
    "customers_pkey" PRIMARY KEY, btree (customer_id)
    "customers_email_key" UNIQUE CONSTRAINT, btree (email)
Referenced by:
    TABLE "addresses" CONSTRAINT "addresses_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    TABLE "orders" CONSTRAINT "orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    TABLE "reviews" CONSTRAINT "reviews_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)

project=> \d deliveries

      Table "public.deliveries"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
delivery_id | integer                |           | not null | nextval('deliveries_delivery_id_seq'::regclass)
order_id    | integer                |           |           |
agent_id    | integer                |           |           |
delivery_status | character varying(50) |           |           |
delivery_time | time without time zone |           |           |
Indexes:
    "deliveries_pkey" PRIMARY KEY, btree (delivery_id)
Foreign-key constraints:
    "deliveries_agent_id_fkey" FOREIGN KEY (agent_id) REFERENCES delivery_agents(agent_id)
    "deliveries_order_id_fkey" FOREIGN KEY (order_id) REFERENCES orders(order_id)

project=> \d delivery_agents

      Table "public.delivery_agents"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
agent_id | integer                |           | not null | nextval('delivery_agents_agent_id_seq'::regclass)
name     | character varying(100) |           |           |
phone    | character varying(20)  |           |           |
vehicle_number | character varying(50) |           |           |
Indexes:
    "delivery_agents_pkey" PRIMARY KEY, btree (agent_id)
Referenced by:
    TABLE "deliveries" CONSTRAINT "deliveries_agent_id_fkey" FOREIGN KEY (agent_id) REFERENCES delivery_agents(agent_id)

project=> \d food_categories

      Table "public.food_categories"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
category_id | integer                |           | not null | nextval('food_categories_category_id_seq'::regclass)
category_name | character varying(100) |           |           |
Indexes:
    "food_categories_pkey" PRIMARY KEY, btree (category_id)
Referenced by:
    TABLE "menu_items" CONSTRAINT "menu_items_category_id_fkey" FOREIGN KEY (category_id) REFERENCES food_categories(category_id)
    TABLE "restaurant_categories" CONSTRAINT "restaurant_categories_category_id_fkey" FOREIGN KEY (category_id) REFERENCES food_categories(category_id)

project=> \d food_categories

      Table "public.food_categories"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
category_id | integer                |           | not null | nextval('food_categories_category_id_seq'::regclass)
category_name | character varying(100) |           |           |
Indexes:
    "food_categories_pkey" PRIMARY KEY, btree (category_id)
Referenced by:
    TABLE "menu_items" CONSTRAINT "menu_items_category_id_fkey" FOREIGN KEY (category_id) REFERENCES food_categories(category_id)
    TABLE "restaurant_categories" CONSTRAINT "restaurant_categories_category_id_fkey" FOREIGN KEY (category_id) REFERENCES food_categories(category_id)

project=> \d menu_items

      Table "public.menu_items"
  Column |          Type          | Collation | Nullable |          Default
-----|-----|-----|-----|-----
item_id | integer                |           | not null | nextval('menu_items_item_id_seq'::regclass)
restaurant_id | integer                |           |           |
category_id | integer                |           |           |
item_name  | character varying(100) |           |           |
description | text                  |           |           |
price      | numeric(10,2)          |           |           |
availability | boolean                |           | true      |
Indexes:
    "menu_items_pkey" PRIMARY KEY, btree (item_id)
Foreign-key constraints:
    "menu_items_category_id_fkey" FOREIGN KEY (category_id) REFERENCES food_categories(category_id)
    "menu_items_restaurant_id_fkey" FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
Referenced by:
    TABLE "order_items" CONSTRAINT "order_items_item_id_fkey" FOREIGN KEY (item_id) REFERENCES menu_items(item_id)

```

```

project=> \d order_items
Did not find any relation named "order".
project=> \d order_items

      Table "public.order_items"
  Column |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
order_item_id | integer         |           | not null | nextval('order_items_order_item_id_seq'::regclass)
order_id      | integer         |           |          |
item_id       | integer         |           |          |
quantity      | integer         |           |          |
price         | numeric(10,2)   |           |          |

Indexes:
    "order_items_pkey" PRIMARY KEY, btree (order_item_id)
Foreign-key constraints:
    "order_items_item_id_fkey" FOREIGN KEY (item_id) REFERENCES menu_items(item_id)
    "order_items_order_id_fkey" FOREIGN KEY (order_id) REFERENCES orders(order_id)

project=> \d orders

      Table "public.orders"
  Column |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
order_id | integer         |           | not null | nextval('orders_order_id_seq'::regclass)
customer_id | integer         |           |          |
order_date | date            |           |          | CURRENT_TIMESTAMP
status    | character varying(50) |           |          |
total_amount | numeric(10,2)   |           |          |

Indexes:
    "orders_pkey" PRIMARY KEY, btree (order_id)
Foreign-key constraints:
    "orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
Referenced by:
    TABLE "deliveries" CONSTRAINT "deliveries_order_id_fkey" FOREIGN KEY (order_id) REFERENCES orders(order_id)
    TABLE "order_items" CONSTRAINT "order_items_order_id_fkey" FOREIGN KEY (order_id) REFERENCES orders(order_id)
    TABLE "payments" CONSTRAINT "payments_order_id_fkey" FOREIGN KEY (order_id) REFERENCES orders(order_id)

project=> \d payments

      Table "public.payments"
  Column |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
payment_id | integer         |           | not null | nextval('payments_payment_id_seq'::regclass)
order_id   | integer         |           |          |
payment_date | date            |           |          | CURRENT_TIMESTAMP
amount     | numeric(10,2)   |           |          |
payment_method | character varying(50) |           |          |
payment_status | character varying(50) |           |          |

Indexes:
    "payments_pkey" PRIMARY KEY, btree (payment_id)
Foreign-key constraints:
    "payments_order_id_fkey" FOREIGN KEY (order_id) REFERENCES orders(order_id)

```

```

project=> \d restaurant_categories
Table "public.restaurant_categories"
  Column      | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
restaurant_id | integer |           | not null |
category_id   | integer |           | not null |
Indexes:
    "restaurant_categories_pkey" PRIMARY KEY, btree (restaurant_id, category_id)
Foreign-key constraints:
    "restaurant_categories_category_id_fkey" FOREIGN KEY (category_id) REFERENCES food_categories(category_id)
    "restaurant_categories_restaurant_id_fkey" FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)

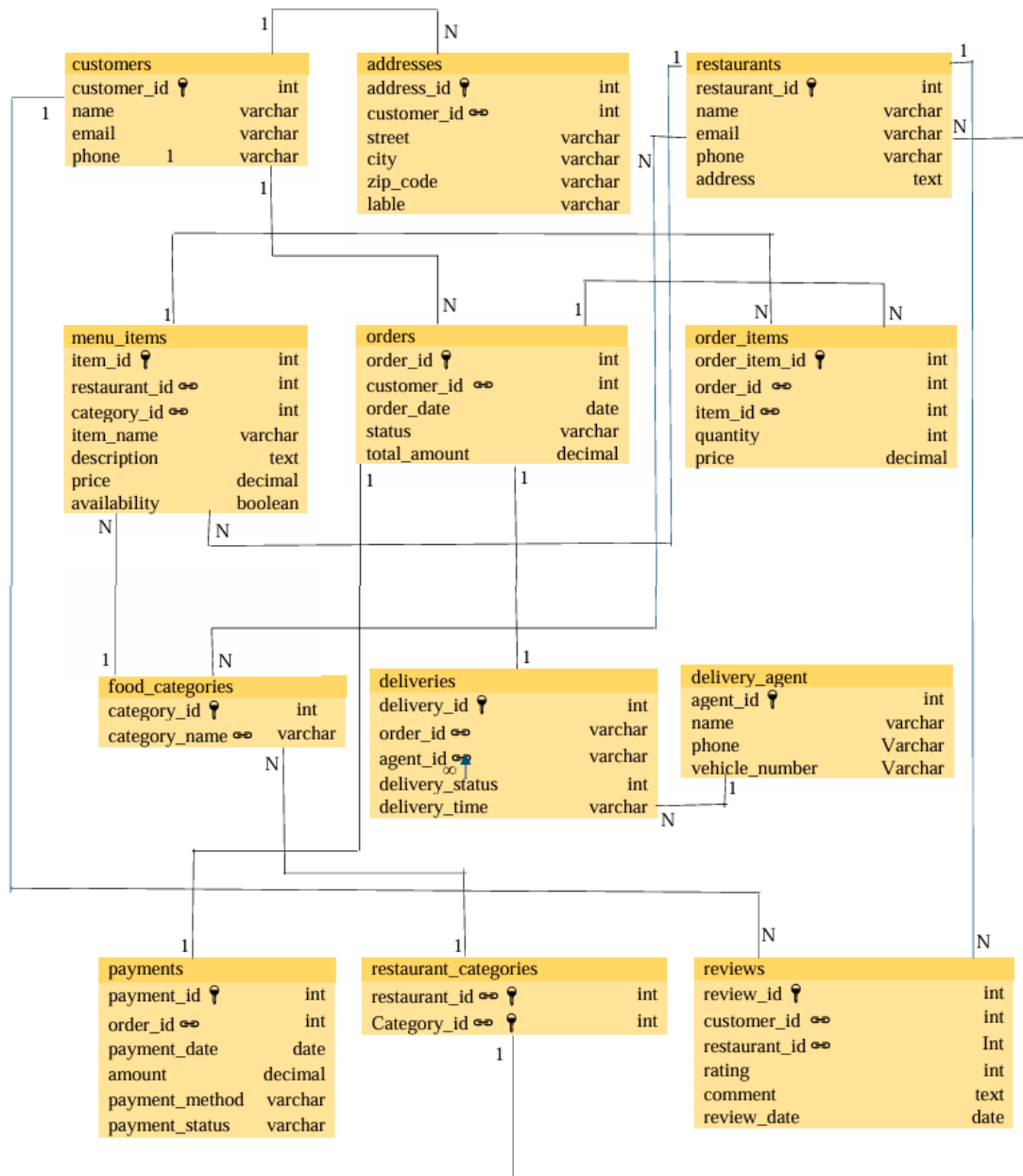
project=> \d restaurants
Table "public.restaurants"
  Column      | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
restaurant_id | integer       |           | not null | nextval('restaurants_restaurant_id_seq'::regclass)
name          | character varying(100) |           |          |
email         | character varying(100) |           |          |
phone         | character varying(20)  |           |          |
address       | text          |           |          |
Indexes:
    "restaurants_pkey" PRIMARY KEY, btree (restaurant_id)
Referenced by:
    TABLE "menu_items" CONSTRAINT "menu_items_restaurant_id_fkey" FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
    TABLE "restaurant_categories" CONSTRAINT "restaurant_categories_restaurant_id_fkey" FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
    TABLE "reviews" CONSTRAINT "reviews_restaurant_id_fkey" FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)

project=> \d reviews
Table "public.reviews"
  Column      | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
review_id     | integer |           | not null | nextval('reviews_review_id_seq'::regclass)
customer_id   | integer |           |          |
restaurant_id | integer |           |          |
rating        | integer |           |          |
comment       | text    |           |          |
review_date   | date    |           |          | CURRENT_TIMESTAMP
Indexes:
    "reviews_pkey" PRIMARY KEY, btree (review_id)
Check constraints:
    "reviews_rating_check" CHECK (rating >= 1 AND rating <= 5)
Foreign-key constraints:
    "reviews_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    "reviews_restaurant_id_fkey" FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)

```

Entity Relationship Diagram (ERD)

The system consists of 12 core tables: Customers, Addresses, Restaurants, Menu Items, Food Categories, Restaurant Categories, Orders, Order Items, Payments, Reviews, Deliveries, and Delivery Agents. Relationships among the tables are well defined and follow 3NF normalization.



Core Functionalities of BiteDash - Food Delivery Management System

1. Database Management System

Comprehensive Database Schema

- **12 interconnected tables** modeling all aspects of food delivery operations
- **Relational database design** with proper primary and foreign key relationships
- **Normalized structure** to minimize data redundancy
- **Composite primary keys** for junction tables (e.g., restaurant_categories)

Tables Included:

- Customers, Addresses, Restaurants, Food Categories, Menu Items
- Orders, Order Items, Payments, Delivery Agents, Deliveries, Reviews
- Restaurant-Categories relationship table

2. API Backend

For Each Table (12 tables × 4 operations = 48 endpoints):

- **GET All Records:** Retrieve complete dataset from any table
- **GET Count:** Fetch total record count for each table
- **POST Insert:** Add new records to any table
- **DELETE:** Remove records by primary key

API Features:

- **Standardized endpoints** following REST conventions
- **Proper error handling** for database operations
- **Input validation** before database operations
- **Foreign key constraint enforcement** at API level
- **JSON payload support** for all requests/responses

3. Frontend Dashboard

User Interface Components:

- **Responsive design** that works on desktop and mobile devices
- **Modern UI/UX** with intuitive navigation and visual hierarchy
- **Interactive elements** with hover effects and transitions

Core Functional Modules:

A. Data Visualization Dashboard

- **Table cards** showing record counts with icons
- **Quick access** to all database tables
- **One-click access** to full table data

B. Data Insertion Module

- **Dynamic form generation** based on table schema
- **Automatic field detection** (primary keys, data types)
- **Input validation** with user guidance
- **Success/error feedback** for operations

C. Data Deletion Module

- **Safe deletion interface** with confirmation dialogs
- **Support for simple and composite primary keys**
- **Warning messages** for potentially dangerous operations
- **Referential integrity checks**

D. Analytics & Reporting

- **Interactive charts** using Chart.js
- **Multiple visualization types** (pie, bar, line, polar area)
- **Report previews** on dashboard cards

- **Detailed modal views** with additional statistics

E. System Information

- **About section** with project description
- **Feature highlights** with icons
- **Technical stack information**
- **Responsive footer** with navigation links

4. Technical Implementation

Frontend Technologies:

- **HTML5** semantic markup
- **CSS3** with modern layout (Flexbox, Grid)
- **JavaScript** for dynamic functionality
- **Chart.js** for data visualization
- **Responsive design** with media queries

Backend Technologies:

- **Neon.tech** serverless PostgreSQL database
- **API** implementation on Codespaces
- **Proper HTTP methods** (GET, POST, DELETE)

5. Advanced Features

User Experience Enhancements:

- **Loading states** for asynchronous operations
- **Success/error notifications** for user feedback
- **Form validation** with helpful messages
- **Responsive navigation** with hamburger menu

Data Management:

- **Foreign key handling** in forms
- **Auto-generated fields** detection (SERIAL PKs)
- **Data type indications** for each field
- **Bulk data viewing** in separate windows

Analytics Capabilities:

- **Multiple chart types** for different data perspectives
- **Color-coded data series** for better readability
- **Interactive chart elements** (hover details)
- **Statistical summaries** alongside visualizations

6. Security Considerations

- **API endpoint protection** (though not shown in code)
- **Input sanitization** to prevent SQL injection
- **Proper HTTP methods** for each operation type
- **Confirmation dialogs** for destructive operations

7. Deployment Architecture

- **Frontend:** Static hosting (can be deployed anywhere)
- **Backend:** Codespaces-hosted API endpoints
- **Database:** Neon.tech serverless PostgreSQL
- **CORS management** for cross-origin requests

8. Documentation Features

- **Self-documenting UI** with field type information
- **Tooltips and helper text** throughout interface
- **Code comments** explaining complex logic

API's method used in project

There are four types of API methods used in Bite Dash food ordering system

1. GET All Records API

- **Function:** Retrieves all data rows from a specific table
- **Example:** /customers returns all customer records

Customers

customer id	name	email	phone
1	Ahmed Raza	ahmed.raza@example.com	0300-1234567
2	Bilal Khan	bilal.khan@example.com	0301-2345678
3	Fahad Sheikh	fahad.sheikh@example.com	0302-3456789
4	Usman Tariq	usman.tariq@example.com	0303-4567890
5	Zeeshan Mehmood	zeeshan.mehmood@example.com	0304-5678901
6	Imran Abbas	imran.abbas@example.com	0305-6789012
7	Adeel Javed	adeel.javed@example.com	0306-7890123
8	Noman Iqbal	noman.iqbal@example.com	0307-8901234
9	Saad Farooq	saad.farooq@example.com	0308-9012345
10	Rizwan Ahmed	rizwan.ahmed@example.com	0309-0123456
11	Tariq Mehmood	tariq.mehmood@example.com	0310-1123456
12	Junaid Akram	junaid.akram@example.com	0311-2234567
13	Owais Anwar	owais.anwar@example.com	0312-3345678

Code:

```
app.get('/customers', async (req, res) => {  
  try {  
    const result = await pool.query('SELECT * FROM customers');  
    res.json(result.rows);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

2. GET Count API

- **Function:** Returns the total number of records in a table
- **Example:** count/customers returns {"count":52}



Customers

52 Records

Manage customer records and
profiles

Code:

```
app.get('/count/customers', async (req, res) => {  
  try {  
    const result = await pool.query('SELECT COUNT(*) FROM customers');  
    res.json(result.rows[0]);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

3. POST Insert API

- **Function:** Adds a new record to a table
- **Example:** /add-customer with customer data in body

Insert New Record

 Select Table

Customers 

 name

Customer full name

Type: VARCHAR(100)

 email

Customer email (unique)

Code:

```
app.post('/add-customer', async (req, res) => {
  const { name, email, phone } = req.body;

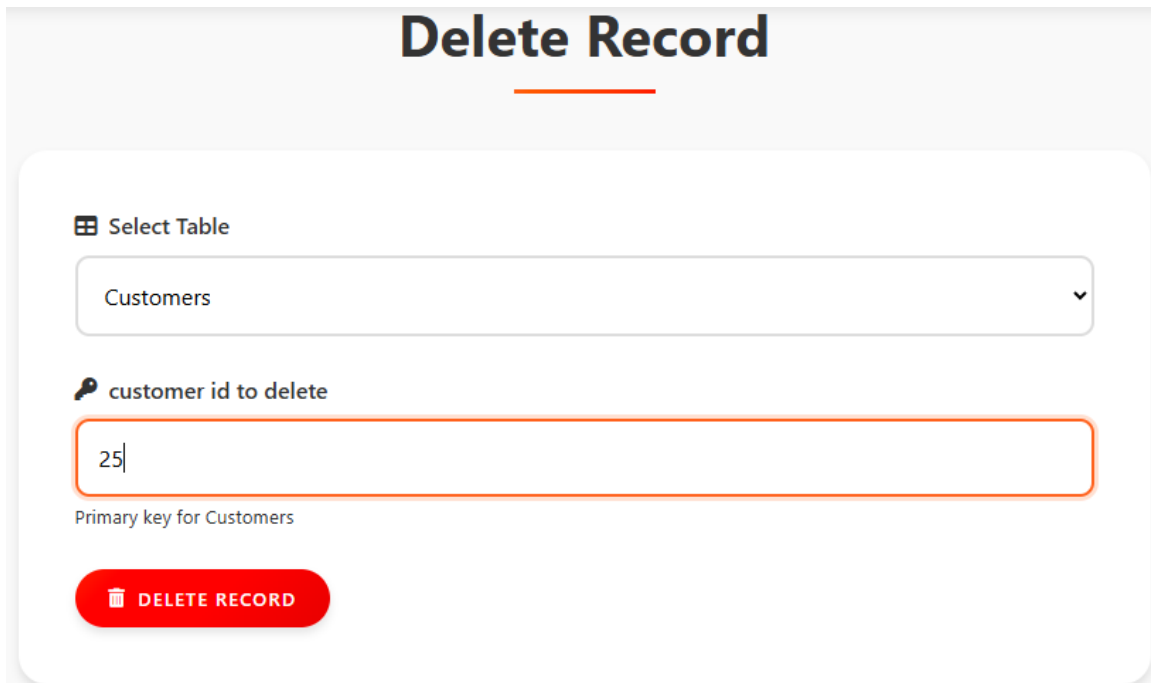
  if (!name || !email) {
    return res.status(400).json({ error: 'Name and email are required' });
  }

  try {
    const result = await pool.query(
      'INSERT INTO customers (name, email, phone) VALUES ($1, $2, $3) RETURNING *',
      [name, email, phone]
    );
    res.status(201).json({
      message: 'Customer added successfully!',
      data: result.rows[0]
    });
  } catch (err) {
    if (err.code === '23505') { // Unique violation
      return res.status(400).json({ error: 'Email already exists' });
    }
    res.status(500).json({ error: err.message });
  }
});
```

```
}  
});
```

4. DELETE API

- **Function:** Removes a record by its primary key
- **Example:** /delete-customer-id: removes customer with ID



The image shows a web form titled "Delete Record" with a red underline. Below the title is a section labeled "Select Table" with a dropdown menu showing "Customers". Below that is a label "customer id to delete" with a key icon, followed by a text input field containing "25". Below the input field is the text "Primary key for Customers". At the bottom is a red button with a trash icon and the text "DELETE RECORD".

Code:

```
app.delete('/delete-customer/:id', async (req, res) => {  
  try {  
    // First check if customer exists  
    const check = await pool.query(  
      'SELECT * FROM customers WHERE customer_id = $1',  
      [req.params.id]  
    );  
  
    if (check.rowCount === 0) {  
      return res.status(404).json({ error: 'Customer not found' });  
    }  
  
    // Delete customer
```

```

const result = await pool.query(
  'DELETE FROM customers WHERE customer_id = $1 RETURNING *',
  [req.params.id]
);

res.json({
  message: 'Customer deleted successfully!',
  deleted: result.rows[0]
});
} catch (err) {
  if (err.code === '23503') {
    return res.status(400).json({
      error: 'Cannot delete customer as they have associated records
(addresses, orders, etc.)'
    });
  }
  res.status(500).json({ error: err.message });
}
});

```

Bite Dash Front-End Screenshots & Interface

The image displays two screenshots of the BiteDash web application interface.

Top Screenshot: BiteDash – Online Food Delivery System

The header features the BiteDash logo and navigation links: Home, Tables, Insert, Delete, Reports, and About. The main banner includes an illustration of a hand holding a smartphone with a food ordering app, a delivery person on a red scooter, and a customer at a door. The text reads: "BiteDash – Online Food Delivery System" and "Smart, Fast & Delicious! Manage every part of your food delivery chain from one dashboard." A prominent orange button labeled "GET STARTED" is centered below the banner.

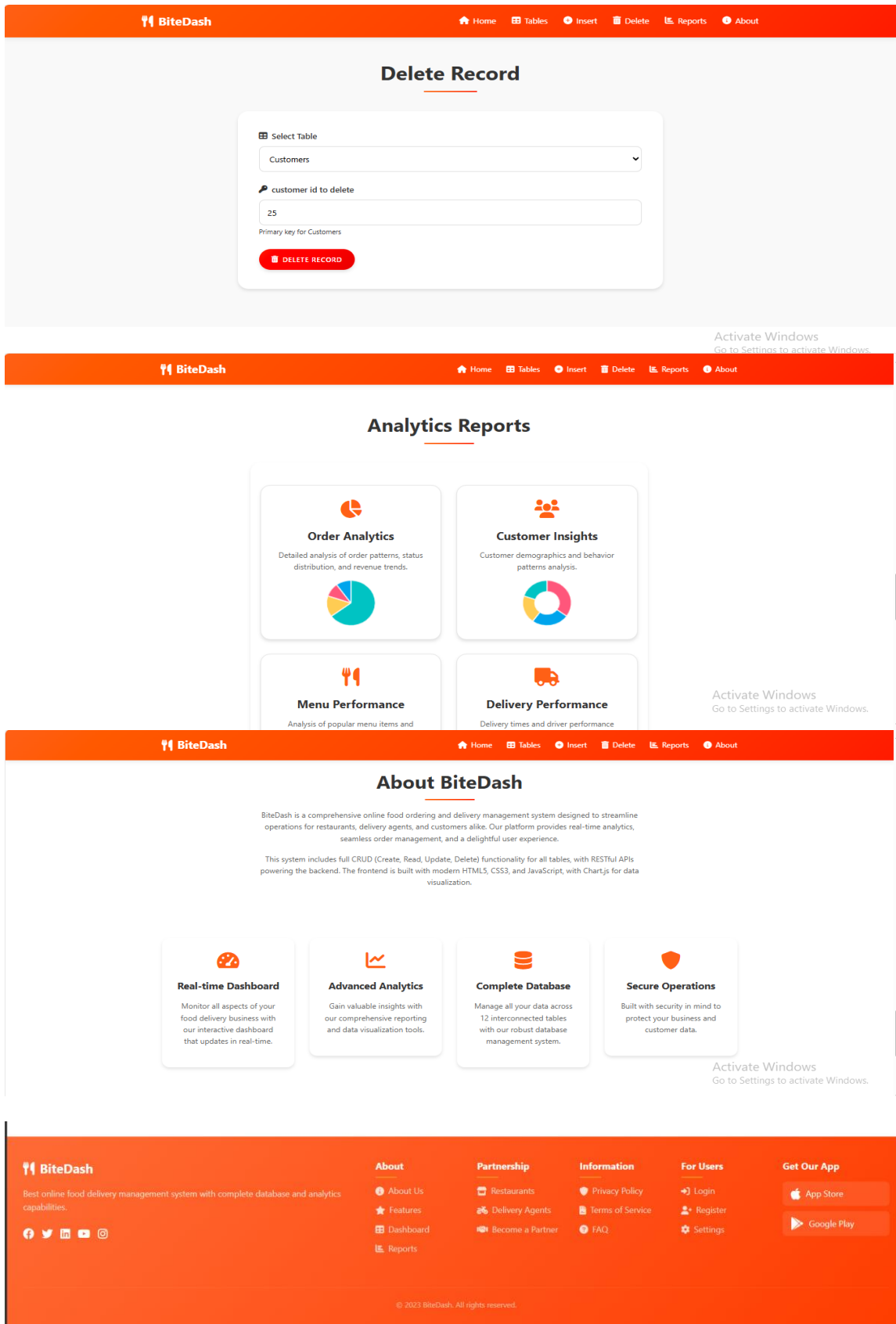
Dashboard Tables

The dashboard displays eight tables, each with an icon, title, record count, and a brief description:

- Customers** (52 Records): Manage customer records and profiles
- Addresses** (63 Records): Manage customer delivery addresses
- Restaurants** (14 Records): Manage restaurant partners and information
- Food Categories** (7 Records): Manage food categories and types
- Menu Items** (67 Records): Manage restaurant menu items and pricing
- Orders** (81 Records): Manage customer orders and status
- Order Items** (218 Records): Manage individual items within orders
- Payments** (43 Records): Manage payment records and transactions

Bottom Screenshot: Insert New Record

The "Insert New Record" form includes a "Select Table" dropdown menu set to "Customers". Below this are input fields for "name" (Customer full name, Type: VARCHAR(100)), "email" (Customer email (unique), Type: VARCHAR(100)), and "phone" (Customer phone number, Type: VARCHAR(20)). An orange "INSERT DATA" button is at the bottom. A watermark "Activate Windows Go to Settings to activate Windows." is visible in the background.



Conclusion

The Online Food Delivery System project successfully demonstrates a complete end-to-end application integrating database design, backend API development, and frontend interface. By leveraging modern tools such as PostgreSQL, Neon.tech, Node.js, and JavaScript, the system effectively models the real-world workflow of an online food ordering platform. It provides customers with an easy-to-use system while offering scalability and manageability for developers. This project also emphasizes the importance of database normalization, API structure, and user-centric design.